

CHAPTER 5

IMPLEMENTATION AND TESTING

Implementation refers to building the project as planned, including coding, module development, and integration of system components. Testing ensures that the developed project works as expected, identifies errors, and validates performance. Together, these processes confirm that the system is functional, reliable, and meets requirements.

5.1 IMPLEMENTATION APPROACHES

Implementation approaches describe the strategies used to introduce the Self-Drive Car Rental System into real-world usage. The approach must minimize risks, ensure reliability, and achieve smooth adoption by users (car owners and renters).

- **Direct Implementation:** Immediately replacing any manual or existing booking methods with the new system. This is fast but risky, as failures can impact all car owners and customers.
- **Parallel Implementation:** Running both the manual booking process and the digital system simultaneously until the new one proves reliable. This ensures safety but requires more resources.
- **Phased Implementation:** Gradually introducing features such as user registration, car listing, booking, payments, and reporting step by step. This lowers risk and allows continuous improvements.
- **Pilot Implementation:** Launching the system for a small set of owners and renters first. Once tested successfully, it is rolled out to all users. This minimizes risks but may not reveal issues at larger scale.

5.2 CODING DETAILS AND CODE EFFICIENCY

5.2.1 CODING DETAILS

The Self-Drive Car Rental System is developed using:

- Frontend: React/React Native for user interfaces.
- Backend: Node.js APIs with Supabase Edge Functions.
- Database: PostgreSQL/Supabase for secure data storage.

Key coding practices:

- Code is organized into modules (authentication, car listing, booking, payments, admin).
- Variables and functions use consistent naming conventions.
- Comments are included for readability and debugging.
- Authentication and role management handled using Supabase Auth.
- Payment services integrated as reusable modules.
- Modular design improves maintainability, scalability, and extension of features.

5.2.2 CODE EFFICIENCY

- Optimized SQL queries for booking, payments, and availability checks.
- UUIDs used for user, car, and booking IDs to avoid duplication.
- Asynchronous operations (e.g., booking confirmation, payment processing) improve speed.
- Minimal API calls reduce load and increase responsiveness.
- Reusable React components minimize redundancy.
- Efficient coding ensures faster performance, reduced costs, and reliable operations.

5.3 TESTING APPROACH**5.3.1 UNIT TESTING**

- Each module tested individually.
- Example: Checking booking cost calculation without system integration.

5.3.2 INTEGRATION TESTING

- Modules combined and tested together.
- Example: Car availability linked with payment processing and booking confirmation.

5.3.3 SYSTEM TESTING

- Entire system tested end-to-end.
- Workflows verified: user registration → car listing → booking → payment → confirmation.
- Ensured requirements were met and errors fixed before deployment.

5.3.4 USER ACCEPTANCE TESTING (UAT)

- Conducted with real users (car owners and renters).
- Validated system usability, accuracy, and reliability.
- Confirmed the platform solved booking and rental challenges effectively.

5.3.5 BLACK BOX TESTING

- Input/output tested without looking into code.
- Example: Renter enters booking details → System processes → Output is confirmation.

Techniques applied:

- Equivalence Partitioning (valid/invalid booking dates).
- Boundary Value Analysis (minimum/maximum rental duration).
- Decision Table Testing (different payment outcomes).
- Error Guessing (invalid car ID, expired license upload).
- Use Case Testing (real rental scenarios).

5.3.6 WHITE BOX TESTING

- Internal logic and database queries tested.
- Example: Checking car availability queries, discount logic, and authentication flows.

Techniques applied:

- Statement Coverage (every line of code executed)
- Branch Coverage (all decision outcomes tested).
- Condition Coverage (complex conditions in booking/payment tested).
- Path Coverage (all code execution paths validated).

5.4 MODIFICATIONS AND IMPROVEMENTS

To maintain quality, several modifications and improvements were applied after initial development.

Modifications:

- Fixed issues in booking overlap detection.
- Corrected errors in car availability queries.
- Updated payment integration logic.

Improvements:

- Added multi-factor authentication for secure logins.
- Real-time notifications for booking confirmations and cancellations.
- Advanced reporting for car owners (earnings, booking history).
- Mobile-responsive design for seamless accessibility.

These updates ensure the system is functional, secure, user-friendly, and scalable for future needs.